Архитектурный отчет: Forensic Repair Module

Система прошла путь от использования "тупого" CLI-скрипта к созданию гибридного модуля восстановления данных. Ниже представлено описание текущей архитектуры, которая интегрирована в инфраструктуру ⚙️ INFRA_TOOLS.

1. Концепция гибридного восстановления

Для достижения высокого качества реконструкции юридических документов, поврежденных ошибками кодировки или OCR, мы внедрили двухэтапный конвейер (pipeline):

- **Этап 1: Детерминированные замены (Deterministic Replacement):** Используется конфигурационный JSON-файл forensic_patterns.json. Это позволяет мгновенно исправлять часто встречающиеся ошибки (например, сдвиги символов в именах или названиях законов) без нагрузки на вычислительные ресурсы.
- **Этап 2: Контекстная LLM-реконструкция (Contextual Reconstruction):** Остаточные искажения, которые не поддаются жесткой замене, передаются модели qwen-forensic. Модель работает в режиме строгого "редактора-корректора" с отключенной креативностью (temperature 0.0) для исключения галлюцинаций.

2. Структура инфраструктуры

Компонент	Путь	Назначение
Модуль логики	⚙️ INFRA_TOOLS/forensic_repair.py	Основной Python-скрипт управления конвейером.
База паттернов	⚙️ INFRA_TOOLS/forensic_patterns.json	Конфигурация детерминированных замен.
Логирование	logs/forensic_repair.log	Аудит всех операций восстановления.
Модель	qwen-forensic	Легковесная сущность Ollama с лимитом контекста 1024 токена.

3. Дальнейшие шаги (Roadmap)

1. **Пакетная обработка:** Реализовать поддержку передачи путей к файлам для автоматизации обработки архивов (ожидает подтверждения).
2. **Режим обучения (Auto-Learn):** Реализовать автоматическое пополнение forensic_patterns.json на основе успешных реконструкций, выполненных LLM.
3. **Интеграция с TI-ULA:** Внедрение вызова модуля в основной процесс обработки входящих юридических документов.